

The theoretical analysis of bioinformatic algorithms

An overview and a case study in ANI estimation

Paul Medvedev

Pennsylvania State University

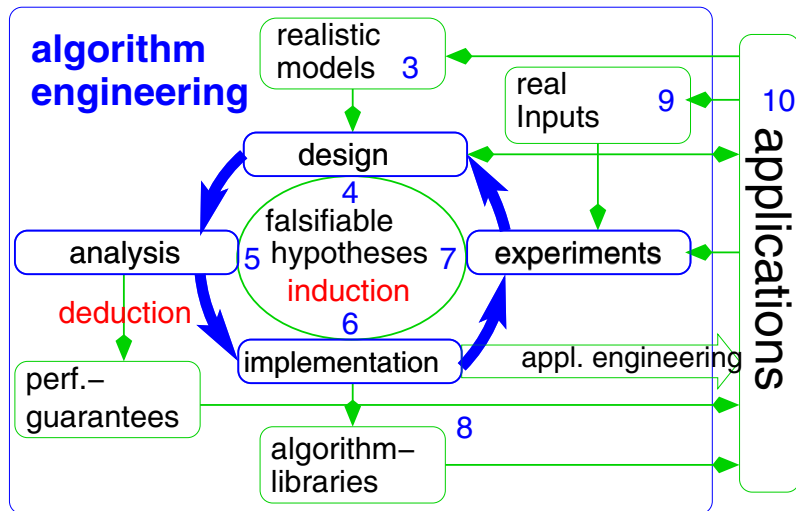
RECOMB 2026

Slides and Papers



Algorithm Engineering

Peter Sanders, *Algorithm Engineering – An Attempt at a Definition* (2009)



Theoretical analysis of algorithm performance

Runtime

- Insertion sort runs in $\mathcal{O}(n^2)$ time
- Expected lookup time in a hash table is $\mathcal{O}(1)$
- Needleman-Wunsch runs in $\Theta(n^2)$ on all inputs.

Space

The BOSS data structure for a de Bruin graph uses exactly $4m + o(m)$ bits.

Accuracy

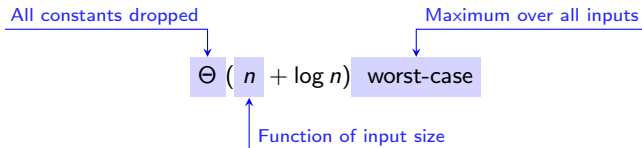
Let $Uni(s, r)$ be the distribution of strings mutated from s with uniform mutation rate r .

$$\Pr_{t \sim Uni(s, r)} \left[\left| \widehat{ANI}(s, t) - ANI(s, t) \right| \leq 0.1r \right] \geq 95\%$$

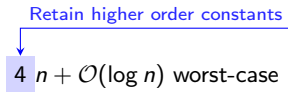
Theoretical **analysis** of algorithm performance

Worst-case analysis

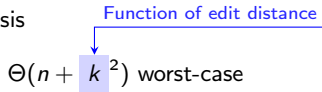
1. Traditional worst-case analysis



2. Retaining higher order constants



3. Parametrized worst-case analysis



Theoretical **analysis** of algorithm performance

Distributional-based analysis

4. Distributional analysis

$$\mathbb{E}_{I \sim D} [\text{Alg}(I)] = \mathcal{O}(n) \qquad \Pr_{I \sim D} [\text{Alg}(I) = \mathcal{O}(n)] \geq 99\%$$

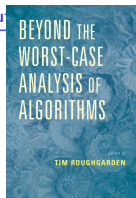
Input drawn from a distribution

5. Semi-random models

Worst-case over adversary's choice

$$\text{For all strings } s, \quad \Pr_{t \sim \text{Uni}(s,r)} [\text{Alg}(I) = \mathcal{O}(n)] \geq 99\%$$

With high probability over distribution



1. Why to do theoretical analysis of performance
2. The goals of theoretical analysis of performance
3. Are the goals achieved in bioinformatics?
 - k -mer data structure space
 - edit distance speed
 - genome assembly accuracy
4. Accuracy of ANI estimation
 - Mash estimator
 - Accuracy of Mash estimator
 - Repeat-aware estimator
 - Accuracy analysis

Why is it important to do?

Sometimes not done!

Review

Hash functions in nucleotide sequence analysis

Ke Chen,^{1,4} Xiang Li,^{1,4} Qian Shi,¹ Mingfu Shao,^{1,2} and Paul Medvedev^{1,2,3}

¹Department of Computer Science and Engineering, The Pennsylvania State University, University Park, Pennsylvania 16802, USA;

²Huck Institutes of the Life Sciences, The Pennsylvania State University, University Park, Pennsylvania 16802, USA; ³Department of Biochemistry and Molecular Biology, The Pennsylvania State University, University Park, Pennsylvania 16802, USA

Goals

Predict the empirical performance of an algorithm

- How will the performance generalize to new datasets?

Design algorithms that perform well in practice

- Be a yardstick to drive performance improvement

Are the goals achieved in bioinformatics?

Different areas

- Phylogeny
- Comparative (whole-genome) genomics
- Algorithms for sequencing data
- Structure prediction

In sequencing data algorithms

- Analysis not wide spread
- “Run-time is $\mathcal{O}(n \log n + nd) \dots \mathcal{O}(n^2)$ in the worst-case but less in practice”
- ILPs, HMMs, etc... are **applied in** but not **developed for** bioinformatics

Roadblocks

- Worst-case analysis can be too pessimistic
- Beyond worst-case analysis is technically challenging
- Low-level run-time optimizations are hard to capture
- Time pressure to develop, analyze, and apply algorithm

Three vignettes to follow

review articles



A case study reveals the theoretical analysis of algorithms is not always as helpful as standard dogma might suggest.

BY PAUL HEDGECOCK

Theoretical Analysis of Sequencing Bioinformatics Algorithms and Beyond

When I was a first-year computer science undergraduate student how to quantify the speed of an algorithm, they look at me puzzled and tell me to just find the algorithm and see how long it takes. Such original analysis, in fact, is the most direct and natural way to measure algorithm performance. However, it has long been understood to have many shortcomings. To overcome these shortcomings, computer scientists developed ways to theoretically analyze algorithm performance. The most common technique for this is traditional worst-case analysis. For example, we say merge sort runs in $\mathcal{O}(n \log n)$ worst-case time, which

usually means that there exists some input such that the time required to solve a problem of size n is at least $\Omega(n \log n)$. This is a pessimistic analysis, because we are interested in analyzing average case performance, not the worst case. Additionally, theoretical analysis can be used to compare two different algorithms. For example, merge sort is usually faster than insertion sort, which is usually faster than bubble sort. However, this is only true for the average case. In the worst case, insertion sort is faster than merge sort.

These three types of case (worst-case, average case, and best case) are all useful in understanding the behavior of an algorithm. However, they are not always as helpful as standard dogma might suggest. In fact, they can be misleading. For example, the worst-case analysis of an algorithm can be very pessimistic. In the worst case, an algorithm can take an exponential amount of time to run. This is not always the case. In many cases, the worst-case analysis is not the most useful.

However, the worst-case analysis is not always the most useful. In many cases, the average-case analysis is more useful. For example, the average-case analysis of an algorithm can be very helpful in understanding its behavior. In fact, the average-case analysis is often the most useful. This is because the average-case analysis is more representative of the real world. In the real world, the input to an algorithm is usually not the worst case. It is usually a random input. This is why the average-case analysis is so useful.

DOI: 10.1093/bioinformatics/btt001

Vignette 1/3: Analysis of space of k -mer data structures

A success story in sequencing bioinformatics

Compact data structures

- Allow efficient queries using as little space as possible
- FM-index: **applied in** but not **developed by** bioinformatics
- Traditional worst-case analysis but retaining leading constants



Data structures for k -mer sets

Data structures based on k -mers for querying large collections of sequencing data sets

Data Structures to Represent a Set of k -long DNA Sequences

RAYAN CHIKHI, Center of Bioinformatics and Biostatistics and Integrative Biology
JAN HOLUB, Department of Theoretical Computer Science, Czech Technical University in Prague
PAUL MEDVEDEV, Center for Computational Biology and Bioinformatics

Camille Marchet,¹ Christina Boucher,² Simon J. Puglisi,³ Paul Medvedev,^{4,5,6}
Mikaël Salson,¹ and Rayan Chikhi⁷

¹Université de Lille, CNRS, CRISAL UMR 9189, F-59000 Lille, France; ²Department of Computer and Information Science and Engineering, University of Florida, Gainesville, Florida 32611, USA; ³Department of Computer Science, University of Helsinki, FI-00014, Helsinki, Finland; ⁴Department of Computer Science, The Pennsylvania State University, University Park, Pennsylvania 16802, USA; ⁵Department of Biochemistry and Molecular Biology, The Pennsylvania State University, University Park, Pennsylvania 16802, USA; ⁶Center for Computational Biology and Bioinformatics, The Pennsylvania State University, University Park, Pennsylvania 16802, USA; ⁷Institut Pasteur & CNRS, C3BI USR 3756, F-75015 Paris, France

Goals met!

A space and time-efficient index for the compacted colored de Bruijn graph

Fatemeh Almodaresi[†], HIRAK SARKAR[†], AVI SRIVASTAVA and ROB PATRO*

Sparse and skew hashing of K -mers

Giulio Ermano Pibiri



SALMON

Vignette 2/3: Analysis of run time of edit distance algorithms

A theory/practice gap

In practice

- Exact
 - banded, bit-parallel, wafefront, etc. . .
 - $\Theta(n^2)$ for evolutionary related sequences ($k = \Theta(n)$)
- Heuristic
 - seed-chain-extend, etc. . .

In theory

- Sub-quadratic worst-case is impossible under SETH
- Approximation algorithms on semi-random models
 - not implemented
 - impractical constants

Shaw and Yu (Genome Research)

- seed-chain-extend takes $\mathcal{O}(n \log n)$ time
- semi-random model analysis

research



To what extent have the techniques for theoretical analysis of edit distance algorithms achieved their goals?

BY PAUL HEDWIG

Theoretical Analysis of Edit Distance Algorithms

have studied the problem starting from as early as 1947 and 1952, but it is very much remains an active topic of research today. For example, Demetrescu et al.'s (2005) landmark work on the complexity of edit distance algorithms achieved their goals?

Demetrescu et al. (2005) have studied the problem starting from as early as 1947 and 1952, but it is very much remains an active topic of research today. For example, Demetrescu et al.'s (2005) landmark work on the complexity of edit distance algorithms achieved their goals?

Demetrescu et al. (2005) have studied the problem starting from as early as 1947 and 1952, but it is very much remains an active topic of research today. For example, Demetrescu et al.'s (2005) landmark work on the complexity of edit distance algorithms achieved their goals?

Demetrescu et al. (2005) have studied the problem starting from as early as 1947 and 1952, but it is very much remains an active topic of research today. For example, Demetrescu et al.'s (2005) landmark work on the complexity of edit distance algorithms achieved their goals?

EDIT DISTANCE—A CLASSICAL PROBLEM IN COMPUTER SCIENCE—HAS RECEIVED ONGOING ATTENTION FROM BOTH PRACTITIONERS AND THEORETICIANS. GIVEN TWO STRINGS A AND B , THE EDIT DISTANCE IS THE MINIMUM NUMBER OF SUBSTITUTIONS, INSERTIONS, AND DELETIONS NEEDED TO TRANSFORM A INTO B . FOR EXAMPLE, THE EDIT DISTANCE BETWEEN agf AND $agple$ IS 3: $agf \rightarrow agpl$ (replace f with p), $agpl \rightarrow agple$ (insert e), $agple \rightarrow agple$ (delete a). THE EDIT DISTANCE PROBLEM IS WIDELY KNOWN, AS IT IS OFTEN TAUGHT AS PART OF THE UNDERGRADUATE CURRICULUM TO ILLUSTRATE TWO-DIMENSIONAL DYNAMIC PROGRAMMING. THEORETICIANS

64 JOURNAL OF THE ACM, VOL. 64, NO. 1, JAN. 2011

Vignette 3/3: Analysis of accuracy of genome assemblers

A failure story

Some attempts

- Measure conditions under which assembler can reconstruct genome (Tse and others)
- Measure % of inferable contigs are reported (“omnitigs”)

In practice

- Most assembler design is driven by accuracy on empirical data (e.g. BUSCO)
- Most assemblers use difficult-to-analyze heuristics

Goals of theoretical analysis

- Design? Mostly no.
- Performance? No

Plenty use of theory ... but NOT to analyze accuracy

Assemblathon2 conclusions for short read assemblers

- Rank of assemblers using experimental evaluation depended on dataset and evaluation metrics/parameters.
- Reviewer: “on any reasonably challenging genome, and with a reasonable amount of sequencing, assemblers neither perform all that well nor do they perform consistently”

Empirical evaluation

Powerful and essential

But not enough

- Incentives favor overfitting the data
- New algorithms needed every time technology changes.

Benchmarks and critical assessments

- Great!
- Not always available

1. Why to do theoretical analysis of performance
2. The goals of theoretical analysis of performance
3. Are the goals achieved in bioinformatics?
 - k -mer data structure space
 - edit distance speed
 - genome assembly accuracy
4. Accuracy of ANI estimation
 - Mash estimator
 - Accuracy of Mash estimator
 - Repeat-aware estimator
 - Accuracy analysis

Theoretical analysis on the ground

Estimating average nucleotide identity

ANI

- percentage of matches in an optimal alignment
- alignment based tools accurate but slow
- *k*-mer based tools scale better

Consider

- RefSeq
 - 54k assembled genomes (2016)
- SRA metagenomes
 - 4.7 million raw read datasets (2026)

What scalable ANI computation achieves

- Clustering
- Build phylogenies
- Better compression
- Similarity search

Mash ANI

Ondov et al (2016) with Binomial modification of Sarmashghi et al (2019)

Mash* ANI estimator

- $\left(\frac{2J}{1+J}\right)^{1/k}$
- J : *Jaccard similarity* is the fraction of k -mers that are shared.

No theoretical analysis of estimator accuracy

- What is the bias?
- What is its variance / error?
- Can we have confidence intervals?
- How will accuracy deteriorate w

Mutate first C to A

Assumptions in Mash derivation

1. substitutions only
2. positions mutate independently
3. equal genome-wide mutation rate r
4. $ANI = 1 - r$
5. no duplicate k -mer in original string
6. a k -mer never mutates to a pre-existing k -mer

C C T A A

→

A C T A A



OK

Formalizing the problem

A good model

- Simple enough to analyze
- Realistic enough to be useful



Briefings in Bioinformatics, 20(4), 2019, 1376–1383

doi: 10.1093/bib/bby083

Advance Access Publication Date: 30 January 2018

Paper

Modeling biological problems in computer science: a case study in genome assembly

Paul Medvedev

Corresponding author. Paul Medvedev, Department of Computer Science and Engineering, Department of Biochemistry and Molecular Biology, Center for Computational Biology and Bioinformatics, 506B Wartik Lab, University Park, PA 16802, USA. E-mail: pashadag@cse.psu.edu

Abstract

Downloaded from
http://bib.oxfordjournals.org/

Key idea

- Assumptions made during derivations are where mistakes creep in
- Integrate assumptions into the probabilistic space

The k -span model

Re-derivation of Mash ANI

K -span model

- Replace sequences with intervals
- No genome, no k -mers

Model definition

- Every position is independently marked with probability r
- k -span: interval of length k
- A k -span is mutated if at least one of its positions is marked
-

$$J_{\text{kspan}} \triangleq \frac{\text{n. of non-mutated } k\text{-spans}}{2(\text{n. of mutated } k\text{-spans}) + \text{n. of non-mutated } k\text{-spans}}$$

Definition (K -span ANI)

$$\widehat{ANI} \triangleq \left(\frac{2J_{\text{kspan}}}{1 + J_{\text{kspan}}} \right)^{1/k}$$

When reality fits
our assumptions,
Mash* and k -span
estimators match!

Confidence Intervals

Theorem (Confidence interval)

Given α and J_{kspan} , our algorithm computes two values ANI_{low} and ANI_{high} such that

$$\lim_{L \rightarrow \infty} \Pr[ANI_{low} < ANI < ANI_{high}] = 1 - \alpha$$

Results

E.Coli, $\alpha = 0.05$

<i>k</i> -mer size	<i>substitution rate</i>			
	<i>0.001</i>	<i>0.01</i>	<i>0.1</i>	<i>0.2</i>
21	0.95	0.94	0.93	0.94
51	0.95	0.95	0.95	NA
100	0.95	0.95	NA	NA

Table: No sketching, varying k

<i>sketch size</i>	<i>substitution rate</i>		
	<i>.05</i>	<i>15</i>	<i>.25</i>
1,000	0.97	0.96	1.00
10,000	0.96	0.96	0.97
100,000	0.94	0.95	0.96

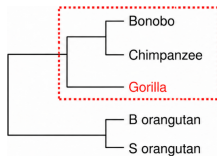
Table: Varying sketch size, fixed $k = 21$

Violation of assumptions

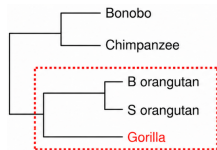
Repeats and centromere regions

α -satellite DNA

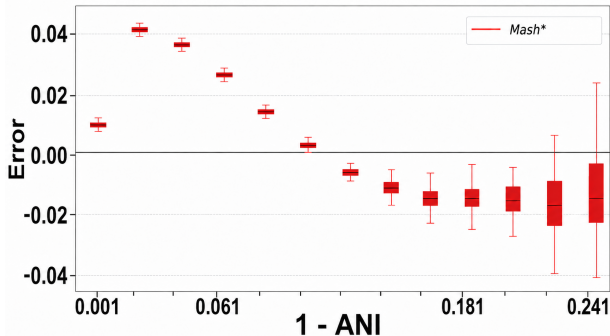
- Highly repetitive
- 100kb from chr21 $\rightarrow$ 4k distinct 30-mers!
- But we assumed no repeats. . .



(a) Correct tree



(b) Mash-based tree



Allowing repeats

Back to k -mers instead of k -spans

Approximations not assumptions

- made as part of deriving an estimator
- not made during accuracy analysis

Accuracy analysis captures the error due to approximations

Assumptions

1. substitutions only
2. positions mutate independently
3. equal genome-wide mutation rate r
4. $ANI = 1 - r$
5. ~~no duplicate k -mer in original string~~
6. ~~A k -mer never mutates to a pre-existing k -mer~~

Approximations

- A k -mer never mutates to a pre-existing k -mer
- (new) occurrences of a k -mer do not overlap each other

Repeat-robust estimator

Core derivation

$\Pr[x \in t] = \Pr\left[\bigcup_{i=1}^L (t_i = x)\right]$

$\approx \Pr\left[\bigcup_{\substack{i \text{ s.t.} \\ s_j = x}} (t_i = x)\right]$

$= 1 - \Pr\left[\bigcap_{\substack{i \text{ s.t.} \\ s_j = x}} (t_i \neq x)\right]$

$\approx 1 - \prod_{\substack{i \text{ s.t.} \\ s_j = x}} \Pr[t_i \neq x]$

$= 1 - \text{occ}(s, x)(1 - (1 - r)^k)$

Repeat-robust estimator

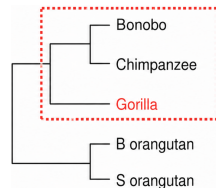
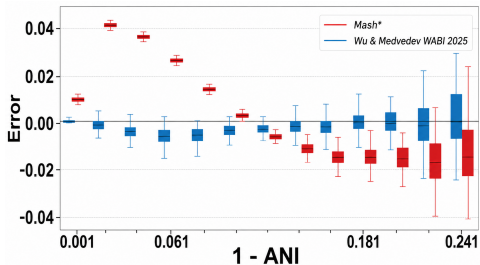
Definition and empirical evaluation

Estimator definition

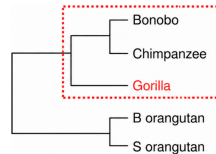
Our estimator is the unique root of

$$S - \sum_i c_i (1 + (1 - \text{ANI}^k)^i) = 0, \text{ where}$$

- S is the n. of shared k -mers
- c_i is the n. of k -mers that occur i times in s .



(c) Wu2025



(a) Correct tree

Bias Theorem

We give an algorithm that computes f_{low} and f_{high} such that

$$f_{low} < |\mathbb{E}[\hat{q}] - q| < f_{high}$$

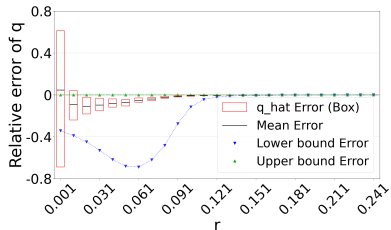
More specifically, f_{low} and f_{high} are functions of $|s|$, k , and

- distribution of k -mer counts in s*
- distribution of pairwise k -mer Hamming distances in s*
- numbers of overlapping k -mer occurrences*

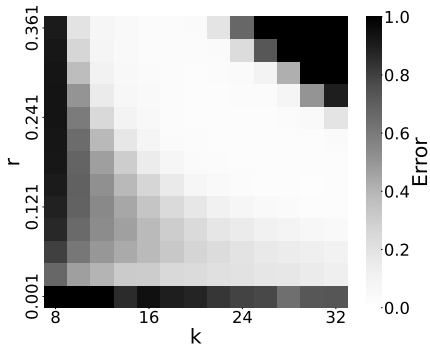
Bias prediction

Empirical results outshine theoretical ones.

Theoretical bias



Empirical error



More to this story

Authors	Venue	Topic
Blanca et al.	RECOMB 2021	<i>K</i> -span model and applications
Belbasi et al	ISMB 2022	Effect of sketching technique
Wu, Blanca, Medvedev	WABI 2025	Repeat-robust estimator
Hera et al.	WABI 2025	Removing the no-indels assumption
Wu & Medvedev	ISMB 2026	Exploiting <i>k</i> -mer count information

- What is theoretical analysis of algorithm performance?
- Goals: prediction and design
- Has it achieved its goals in bioinformatics?
- In algorithms for sequencing data, results are mixed
- ANI estimation
 - Repeat-free
 - Theoretical analysis used to design confidence intervals
 - Repeat-robust
 - Theoretical analysis predicts accuracy of new repeat-robust estimator
 - Supplemented with empirical evaluation



Wrap-up (2/2)

Theoretical analysis of bioniformatic algorithms

- A new research area . . .
- . . . as opposed to a side note in an algorithm paper
- Overarching goal: develop analysis techniques that are simple yet predictive

What can it do?

- Aggregate experiences from different bioinformatics sub-areas
- Find success stories to emulate
- Develop a toolbox of analysis techniques
- Retrospective prediction of algorithm performance
- Formulate best strategies for combining empirical and theoretical analysis



Haonan Wu



Antonio Blanca



David Koslicki



Bob Harris

